Build the Python Learning Pages

Codex-ready product and content implementation specification

Source curriculum: MADERA_SAM_PROPOSAL.pptx (19 slides) | Scope: 32 numbered lessons + optional Python Frameworks extension

Audience: Codex working inside the existing project repository. This is an implementation brief, not a classroom handout.

PURPOSE

Create a high-quality Python course inside the project. INSERT WORKING CODE EDITORS INTO THE RELEVANT SECTIONS OF LESSON PAGES, so students can immediately edit and run the example they just learned. For each lesson, provide the actual discussion content, runnable or transparently simulated examples, guided exercises, independent challenges, quizzes with feedback, and the interface components required to make learning interactive. Reuse existing project architecture and conventions rather than starting a separate app.

SOURCE FIDELITY

The 32 lesson titles, three-level structure, Learn → Practice → Apply model, Python Playground, and final-project concept come from the presentation. Detailed teaching points, code demonstrations, interactions, acceptance tests, information architecture, and engineering safeguards below are implementation specifications added for Codex. The presentation also shows “Python Frameworks” with a second number 32; preserve it as an OPTIONAL unnumbered extension unless the repository owner explicitly approves a Lesson 33.

HOW TO USE

1. Place this DOCX alongside your repository or paste its content into a Codex task. Do not assume Codex has read the PPTX; this document is self-contained.
2. Start by giving Codex the exact “Master prompt” on the following pages. Keep the lesson-by-lesson requirements available as its source of truth.
3. Codex must inspect the repository first, then implement in small batches while preserving existing routes, design system, backend, dependencies, and user data.
4. Require a final change report with files touched, implemented versus deferred features, how to run, tests passed, and known limitations.

DELIVERY GOAL

A working, responsive course experience with 32 discoverable lessons, meaningful content on each page, interactive practice where execution support exists, reliable assessment/progress behavior, and a final project workflow. Do not substitute placeholder lorem ipsum, empty lesson pages, or cosmetic-only mock buttons.

01 Master prompt for Codex (copy this section)

ROLE AND OBJECTIVE

You are working in an EXISTING software repository. Implement or enhance the Python education module using this specification. Treat the 32-lesson outline in this document as the content source of truth. Your primary deliverable is a functioning product inside this repository, not a newly generated document. Do not overwrite existing features or invent new backend contracts. Inspect the codebase first.

Mandatory first actions

- Inspect project directories, README, package manifests, app routing, UI components, styling tokens, database models, authentication/roles, current lesson or coding-playground code, testing setup, and deployment conventions.
- Identify the stack and existing implementation: what can be reused, what is missing, and whether in-browser Python execution or isolated server execution already exists.
- Map the course into existing navigation and content storage. If structure is missing, add a data-driven lesson model instead of hardcoding 32 disconnected pages.
- Before changing code, produce a compact implementation plan with planned components, content schema, execution strategy, and test strategy. Then implement rather than stopping at the plan.
- Use the current visual language; make interface decisions fit the project. Do not rebuild authentication, routing, or design system unnecessarily.

Required build result

- Three course categories: Basic Python (01–15), Intermediate Python (16–27), Advanced Python (28–32), with lesson names exactly as listed in Section 04.
- For each lesson page, include a readable introduction, prerequisites, specific learning outcomes, lesson discussion broken into small teachable sections, code examples with explanation, guided practice, independent apply challenge, short scored knowledge check, and lesson completion behavior.
- Shared course overview, lesson listing, lesson detail view, INLINE runnable practice cards inside lesson discussions (not merely a link to a separate playground), optional standalone playground, code output/error view, hints, feedback, progress indicator, and final-project page.
- Exercises must have explicit expected outcomes and deterministic checks where possible. Do not claim autograding if only manual review is implemented.
- At the end provide files changed, test commands and results, sample working routes, unmet requirements and why, and no fabricated implementation claims.

Nonnegotiable engineering rules

- Never run untrusted learner Python source with unrestricted exec/eval in the main application process. Prefer an existing vetted browser runtime or an isolated restricted worker/container with CPU/time, memory, filesystem, and network policies. If execution is not supported, implement a clear gated state and real sample outputs; never pretend execution occurred.
- Do not introduce fake APIs, fake persistence, fake exports, or unsupported external-library execution. Gate or disable unavailable features with an informative state.
- Learner submissions and grades require appropriate authorization and data isolation. Do not expose answers in a way that invalidates grading if assessment integrity matters.
- Never expose secrets, unrestricted network access, host filesystem, internal services, or shell access through the learner playground.
- Honor existing accessibility, i18n, responsive layout, design tokens, linting, test frameworks, dependency policy, and privacy requirements.
- Add unit, integration, and accessible-interaction tests as relevant. Keep content page generation data-driven and testable.

Implementation order

1) Audit existing project. 2) Create lesson schema and manifest. 3) Build shared overview/detail components. 4) Implement lessons 01–15. 5) Implement 16–27. 6) Implement 28–32 with realistic library/runtime constraints. 7) Add quizzes, progress, final project. 8) Test, fix, document. Commit/batch milestones if permitted by the existing workflow.

02 Required screens, page layout, and components

A. Course overview / learning path

- Display the course title, concise purpose, overall progress, and 3 category cards with 15 / 12 / 5 lessons.
- Show ordered lesson cards: number, official title, one-line outcome, completion status, estimated reading/practice only if computed or editor-provided (do not invent precision).
- Allow categories to expand/filter; provide “Continue learning” based on the latest valid progress record or first incomplete lesson.
- Provide a clear route to final project and a course-level completion summary. Respect existing navigation and permission model.

B. Every lesson detail page: content order

5. 1. Header: breadcrumb (Course / Category / Lesson), lesson number and title, category chip, objective, progress status.
6. 2. Before you begin: prerequisites with links to relevant earlier lessons and 2–4 measurable learning outcomes.
7. 3. Learn: 4–7 short explanation blocks with plain language, examples, definitions, warnings, and simple diagrams only when useful.
8. 4. Walkthrough: explain a code snippet, then INSERT AN INLINE EDITOR IMMEDIATELY UNDER THAT SNIPPET, with Run, output and “Try a change” controls where applicable. Show the code/output together; do not route learners away from the lecture.
9. 5. Practice: a second inline guided-task editor if the lesson requires writing different code; otherwise reuse the walkthrough editor without duplicating controls. Include hint ladder, reset, run, and feedback appropriate to implementation.
10. 6. Apply: independent challenge, exact requirements, sample inputs/outputs, edge cases, and clearly stated validation.
11. 7. Check understanding: three or more short questions with answer explanations, one retry path, and visible completion result.
12. 8. Wrap up: three key takeaways, common mistakes, link to prior/next lesson, and a completion button controlled by the defined policy.

C. Shared coding playground

- Editor with line numbers, monospace text, syntax highlighting when available, keyboard access, starter files, reset/revert confirmation, and Run/Stop controls.
- Output surface distinguishing stdout, stderr, tracebacks, timeouts, and unavailable runtime. Do not visually merge these into a single success state.
- Explicit input panel for input() examples; deterministic test inputs for exercise validation; preserve leading/trailing whitespace in outputs where relevant.
- Save code drafts if existing persistence supports it; otherwise clearly label local/ephemeral drafts. Do not falsely promise cross-device saving.
- Multifile support for Modules lesson, a virtual filesystem for File Handling, and plotting preview for Graph/Charts only when technically supported.
- Rate limit/throttle/stop execution as supported. Prevent runaway loops and abusive resource use. Protect against injection and cross-user file exposure.
- On mobile, stack inline editor/output with accessible tabs; never hide essential controls behind hover-only UI. Persist code separately for each lesson block so scrolling or switching lessons does not unexpectedly erase a draft.

C1. Required inline interactive practice behavior - THIS IS THE PRIMARY REQUEST

The learner stays on the lesson detail page and can type, edit, run and inspect code precisely where the lecture introduces the concept. This is an embedded code lab inside the lesson, NOT a separate mandatory playground page. Insert inline practice selectively: when the concept involves executing or changing Python, provide a runnable editor; when the concept is environment setup or a shell-only workflow, provide an appropriate guided checklist or command walkthrough instead.

- Place a compact Practice in this lesson card immediately after the concept/example that it teaches. The preferred flow is: explanation -> annotated code -> editable starter code -> Run -> actual output/error -> a specific change request -> optional Check my work and hint.
- Use one meaningful inline card per lesson by default; add a second only if the lesson covers genuinely distinct tasks (e.g., conditional branches versus loops, or separate chart types). Avoid placing a code editor in every paragraph for decoration.
- Each card includes a descriptive title, 1-2 sentence task, language/mode indicator, accessible multiline code editor, Run (and Stop when supported), Reset with confirmation if draft has changed, Copy, a distinct output panel, and feedback/hint area.
- The Run action must execute the code currently in the editor using the chosen supported Python runtime. Display stdout, stderr, exceptions, input requests, and timeout states faithfully; never show a canned expected output as if the code ran.
- If the example uses input(), provide stdin values or an input interaction. If it uses files or multiple modules, provide a virtual workspace and file list. If it produces a Matplotlib/Seaborn/Pandas/Plotly chart, provide a real plot preview only with a compatible runtime.
- Teacher-provided sample outputs may be shown separately under Expected example output, clearly labeled as reference, not as live execution. Support changing code and rerunning repeatedly without a page refresh.
- Use deterministic Check my work validation on applicable tasks: compare normalized output or invoke isolated test cases. Keep Run and Check separate: Run executes code; Check evaluates whether the task was completed.
- Do not run shell commands like pip install or python -m venv in the Python editor. Use copyable terminal-command examples and checklists for setup lessons, not a fake terminal.
- When no secure Python runner exists, show the code as editable only if there is a real way to run it; otherwise disable Run with explanatory text and provide clear instructions for a supported local alternative. Never present simulated execution as real.
- Lesson completion should require a real attempt at a required inline practice task only where the implementation supports attempts. Optional editors should not prevent completion. Show user progress and drafts accurately.

C2. Sample inline practice card anatomy (visual and functional contract)

```
LEARN: if / elif / else decides which code runs.
WORKED EXAMPLE: show a short explanation and runnable starter.
+-------------------------------------------------------------+
| PRACTICE HERE: Change the score to test another branch      |
| Task: Make the program print "Excellent".                    |
| [ Python editor - learner can type here                   ] |
| score = 78                                                   |
| if score >= 90:                                              |
|     print("Excellent")                                       |
| elif score >= 75:                                            |
|     print("Passed")                                          |
| else:                                                        |
|     print("Keep practicing")                                 |
| [Run code] [Check my work] [Hint] [Reset]                    |
| Actual output (from LAST REAL run): Passed                   |
| Feedback: Change score to 90 or higher, then run again.     |
+-------------------------------------------------------------+
```

Implementation detail: the card above is a wireframe for Codex, not literal layout instructions. Match the existing project UI. Do not hardcode the pictured output: it must be derived from execution and should clear or become marked stale when code changes.

C3. Embedded practice data and runtime contract

```
InlinePracticeBlock {
  id, lessonId, sectionId, title, taskInstructions,
  type: "python-run" | "python-input" | "multifile" | "virtual-file" |
        "sqlite" | "mock-api" | "test-runner" | "plot-preview" |
```

```
        "terminal-guide" | "setup-checklist",
  requiredForCompletion: boolean,
  starterFiles?: [{path, content}], starterCode?: string,
  sampleStdin?: string, expectedOutputLabel?: string,
  hints?: string[], validator?: {mode, publicCases, constraints},
  runtimeRequirements?: string[], supportedFallback?: string
}
CodeRunResult {
  runId, status: "running" | "success" | "error" | "timeout" | "unavailable",
  stdout, stderr, traceback?, artifacts?, elapsedMs?
}
```

- Generate unique practice block IDs, tie them to a stable lesson/section, and mount them in the normal lesson content flow. Code is stored per block; not one global editor state shared across all lessons.
- Keep expected output separate from CodeRunResult. After an edit, visually label any previously displayed result as from the previous run until re-execution.
- Use existing editor/runtime integrations first; choose a suitable isolated execution strategy for this repository after an architecture audit. Browser-based Python and server-isolated Python require different capabilities and threat controls.
- Validate output and challenge logic with reliable actual test executions. Normalize only inconsequential differences such as line-ending style, not meaningful spaces or ordering for ordered outputs.
- Expose plot artifacts through safe, validated media output; avoid injecting untrusted HTML/JS from executed code. Restrict external requests, resource consumption and access to other users data.
- Add tests that execute a Basic sample, input sample, exception sample, file sample, SQL sample, mock-API sample, test-runner sample, and plot sample where supported.

C4. Lesson-by-lesson policy: where to embed code execution

Required = embed a real Python editor/Run interaction. Guided instead = show checklist/terminal instructions, not pretend-execute commands. Conditional = embed only if the runtime safely supports the topic; otherwise explain and use a properly labeled static/reference preview. The detailed task for each lesson appears in Section 05.

Lessons	Interaction policy	Specific notes
01, 03-15	Required: inline Python Run	Core print, expressions, conditions, loops, functions, strings and collections.
02	Guided instead	Local installation/IDE and version-check checklist; optional genuine Hello World browser run.
16	Conditional: multifile	Real file tabs + import only with safe multifile runtime; otherwise executable standard-library import.
17	Conditional: sandbox files	Real virtual files; no access to application host filesystem.
18-26	Required: inline Python Run	Exception handling, recursion, algorithms, OOP, generators, functional tools, regex, dates.
27	Guided instead	Shell/pip and venv commands are explanatory; optional genuine Python import check.
28	Conditional: SQLite	Real isolated SQLite, preferably in-memory and reset between runs.
29	Required: mock JSON lab	Runnable JSON parsing; optional HTTP only via safe mock endpoint.
30	Required: test runner	Run Python asserts/test cases; no real deployment from learner editor.
31	Conditional: Matplotlib	Real image preview if plotting library exists; clearly labeled fallback if absent.
32	Conditional: chart libraries	Library tabs (Pandas, Seaborn, Plotly); render only supported libraries.

D. Assessment and progress

- Quiz each lesson: at least 3 valid questions, correct answer(s), distractors when multiple-choice, explanation for every answer, and one meaningful remediation hint.
- Progress state per lesson: not_started / in_progress / completed; keep a separate latest code draft and quiz attempt where persistence exists.
- Use an explicit completion policy. Recommended: learner views required content, completes practice OR submits required attempt, and meets a configured quiz pass threshold. Do not silently mark complete just for opening page.
- For coding challenges use visible sample tests plus hidden or server-validated tests only if trustworthy isolation and assessment backend exist. Otherwise use transparent self-check rubrics.

- Course progress = completed numbered lessons / 32. Optional Frameworks extension does not increase the denominator by default.
- Use durable user-scoped storage only if the existing project supports it; specify anonymous session-only fallback if not.

E. UX, accessibility, and content polish

- Use consistent visual hierarchy, readable line length, whitespace, code copy controls, descriptive error/hint text, focus outlines, contrast, and semantic headings.
- All major interactions must be operable by keyboard; label inputs and code panes; provide text alternatives for charts and diagrams.
- On incorrect quiz attempts show an explanation, not just a red X; avoid revealing all assessment answers before learner has attempted when grades matter.
- No dead “Run”, “Check”, “Save”, “Next”, “Download”, or “Submit” buttons. Hide or disable with reasons if backend is missing.
- Loading, error, empty state, retry, and success states are required for any actual async operation.
- Use a responsive layout appropriate for phone, tablet, and desktop; navigation must retain orientation and back paths.

03 Content model, implementation contract, and QA

Recommended content structure (adapt field names to the repository; do not impose a new database if structured data already exists):

```
Lesson {
  id, slug, number, category, title, shortSummary,
  prerequisites[], learningObjectives[], sections[],
  examples[{title, language, files?, code, expectedOutput?, explanation, tryChanging}],
  inlinePracticeBlocks[InlinePracticeBlock],
  practice[{instructions, starterCode, hints[], sampleTests[], feedback}],
  applyChallenge{requirements[], starterCode, sampleInputs?, sampleOutputs?, rubric[]},
  quiz[{id, type, question, choices?, correctAnswer, explanation}],
  keyTakeaways[], commonMistakes[], runtimeRequirements[], referenceLinks[]
}
LearnerProgress { userIdOrSessionId, lessonId, status, quizAttempts?, lastCode?, updatedAt }
```

Lesson data rules

- Use exact lesson numbers, section membership, and titles. Stable route slug can be machine-friendly but display title must match curriculum.
- Use 2–4 concise outcomes, 4–7 detailed discussion points, a correct worked example, a guided task, an independent challenge, and three quiz entries per page.
- Every worked snippet must be syntax-checked and its expected output verified where deterministic. Mark set ordering, randomness, floating point, wall time, and external responses as non-deterministic.
- Prefer Python standard library in Basic/Intermediate. Advanced visualization requires declared optional third-party dependencies.
- Starter code should relate directly to the discussion; tasks must be solvable using skills taught so far, except clearly identified optional stretch tasks.
- Each lesson needs a clear success condition, not merely “understand topic” or “practice code.”

Required implementation/test acceptance matrix

Area	Acceptance test
Navigation	All 32 lesson routes resolve; section counts equal 15/12/5; previous/next links correct.
Lesson content	Every lesson includes objective, discussion, code, explanation, practice, apply, quiz, takeaway.
Runner	A learner edits and runs code INLINE on an actual lesson page; output updates from the last real run; errors and timeouts are accurate.
Inputs/Files	input() scenarios and virtual files either work in sandbox or are explicitly marked unavailable.
Quizzes	Correct and wrong answers produce accurate feedback; scores reset/retry correctly.
Progress	Completion saves correctly per user/session, reloads, and overall fraction is out of 32.

Security	Untrusted code does not access host filesystem, secrets, services, or unrestricted network.
Advanced	SQL, APIs, Matplotlib/Pandas/Seaborn/Plotly have honest working or fallback modes.
Accessibility	Keyboard navigation, focus, labels, contrast, chart descriptions and error text pass review.
Responsive	Overview and lesson pages usable on small-screen and desktop widths.
Docs	Setup instructions, dependency list, test commands, known environment limitations included.

Engineering completion checklist

- Lint and type-check where tooling exists. Run build, unit tests, integration checks, and relevant browser/e2e tests; include commands and real results.
- Add lesson manifest integrity tests for unique IDs/slugs, sequential numbering, prerequisites, and no missing content sections.
- Smoke-test INLINE editor behavior on lessons 1, 7, 17, 23, 28, 29, 31, 32 plus at least one quiz, progress update, failed execution, reset, second run, and draft preservation.
- Check Python examples: parse all snippets; run deterministic samples in safe test environment; assert known outputs or mark them variable.
- Maintain backwards compatibility with existing project data and routes; include a migration plan only if schema change is necessary.
- At completion disclose exact remaining work and any unavailable runtime capabilities. Do not mark features complete based solely on UI mockups.

04 Authoritative curriculum outline (from the PPTX)

Slides 2–16 define the lesson list and short teaching intent. The detailed implementation that follows elaborates those titles; it is not a verbatim transcription of the presentation.

BASIC (01–15)	INTERMEDIATE (16–27)	ADVANCED (28–32)
01 Introduction to Python Programming	16 Modules & Packages	28 Database Integration
02 Python Installation	17 File Handling	29 APIs & Web Data
03 Python Syntax	18 Exception Handling	30 Testing & Deployment
04 Variables & Constants	19 Recursion	31 Python Graphs: Matplotlib
05 Data Types	20 Algorithms	32 Python Charts: Seaborn, Plotly & Pandas
06 Operators & Expressions	21 Object-Oriented Programming	
07 Input & Output	22 Constructors	
08 Conditional Statements	23 OOP Principles	
09 Loops & Iteration	24 Advanced Python Tools	
10 Functions	25 Functional Programming	
11 Strings & Text Processing	26 Regex & Date/Time	
12 Lists	27 Package Management	
13 Tuples & Sets		
14 Dictionaries		
15 Nested Data Structures		

Source issue: Advanced slide lists “Python Frameworks (Flask, Django & Pandas Built-in Plotting)” separately but also labels it 32, despite claiming only five advanced subtopics. See Section 06 for an optional frameworks extension. The wording associating Pandas plotting with frameworks is imprecise: Pandas is primarily a data analysis library; Flask and Django are web frameworks.

05 Detailed per-lesson build specifications

Each section below is both a lesson content outline and a development checklist. Codex should build the page in the existing product, not paste this raw specification into the learner-facing screen.

BASIC PYTHON / LESSON 01 OF 32

01 Introduction to Python Programming

Learner outcome Explain what Python is, identify uses, and execute a first program.

Prerequisite Course orientation / no previous coding required

What to discuss on this page

- What a programming language and program are
- Python as a general-purpose interpreted language and what an interpreter does
- Everyday applications: automation, web services, data analysis, AI
- Source code versus program output
- Sequence of instructions and basic debugging mindset
- How lessons, editor, Run, output, and feedback work

Worked example (starter demonstration)

```
print("Hello, Python!")
print("I can write instructions for a computer.")
```

Explain / expected behavior Two print calls run in order and display two lines. Distinguish typing code from running it.

Practice, apply, and evaluate

Guided practice Edit the greeting and add a third print statement about a real-world use of Python.

Independent challenge Create a three-line introduction program and correctly predict the output before running.

Knowledge checks and answer keys

- What is a Python program? => A sequence of instructions written in Python
- What does print() do? => Displays values in standard output
- Name one use of Python. => Automation, websites, data, AI, or another valid use

Page-specific UI / interactive need Welcome example, Run button, output pane, and a short “predict then run” check.

Embedded lab mode python-run

Exact inside-the-lesson hands-on interaction Edit the greeting print statements and Run; show the two real lines from stdout; Check verifies a third printed line.

Inline placement Immediately following the relevant example/discussion, on this same lesson page. Do not make the learner navigate to the standalone playground.

Required completion evidence Working example plus learner practice attempt, application submission or self-check, and quiz result according to the shared completion policy.

BASIC PYTHON / LESSON 02 OF 32

02 Python Installation

Learner outcome Set up or identify a working Python environment and verify it.

Prerequisite Lesson 01 familiarity

What to discuss on this page

- Local Python installation from python.org and compatible operating systems
- Difference between interpreter, terminal, editor, and IDE
- Version verification with python --version or python3 --version
- Create and save a .py file, then run it
- PATH and interpreter selection troubleshooting
- Browser playground alternative that requires no local installation

Worked example (starter demonstration)

```
# save as hello.py and run with python hello.py
print("Python environment ready")
```

Explain / expected behavior The in-page code panel demonstrates the saved script; terminal commands belong in instructions, not Python editor.

Practice, apply, and evaluate

Guided practice Follow an OS-specific checklist to verify Python and run hello.py locally, or use the built-in browser runner.

Independent challenge Submit a short setup report: environment type, verification result, and one issue resolved (or “none”).

Knowledge checks and answer keys

- Does a browser-hosted Python runner require local installation? => No
- What file extension is standard for a Python script? => .py
- What checks the installed Python version? => python --version or python3 --version

Page-specific UI / interactive need OS tabs (Windows/macOS/Linux), copyable shell commands, verification checklist, and local/browser mode distinction.

Embedded lab mode setup-checklist

Exact inside-the-lesson hands-on interaction Show OS-specific installation steps and genuine terminal commands as copyable text. A real browser Hello World demo may be embedded, but never pretend to run python --version in the Python editor.

Inline placement Immediately following the relevant example/discussion, on this same lesson page. Do not make the learner navigate to the standalone playground.

Accuracy / engineering note No version number should be hardcoded as “latest.” Never put shell commands directly into executable Python examples.

Required completion evidence Working example plus learner practice attempt, application submission or self-check, and quiz result according to the shared completion policy.

BASIC PYTHON / LESSON 03 OF 32

03 Python Syntax

Learner outcome Recognize indentation, comments, statements, and basic syntax errors.

Prerequisite Lessons 01–02 as appropriate; directly builds on the previous topic

What to discuss on this page

- Python statements and reading code top to bottom
- Indentation defines code blocks rather than braces
- Colon introduces suites after if, for, def, and similar statements
- Single-line comments with # and meaningful code formatting
- Case sensitivity and identifier spelling
- SyntaxError versus IndentationError and error location

Worked example (starter demonstration)

```
score = 8
if score >= 5:
    print("Pass")
# This comment documents the decision.
```

Explain / expected behavior The print line must be indented under the condition. Explain why a misplaced space can change execution.

Practice, apply, and evaluate

Guided practice Fix a version of the code with missing colon and incorrect indentation.

Independent challenge Given four small snippets, identify which run and repair the broken ones.

Knowledge checks and answer keys

- Why is indentation important? => It groups statements into blocks
- Does Score refer to score? => No, identifiers are case-sensitive
- What begins a one-line comment? => #

Page-specific UI / interactive need Error highlighter with a corrected-code reveal and a visual indentation guide.

Embedded lab mode python-run

Exact inside-the-lesson hands-on interaction Present intentionally broken indentation/colon code; learner repairs and runs it; show actual SyntaxError or successful output.

Inline placement Immediately following the relevant example/discussion, on this same lesson page. Do not make the learner navigate to the standalone playground.

Required completion evidence Working example plus learner practice attempt, application submission or self-check, and quiz result according to the shared completion policy.

BASIC PYTHON / LESSON 04 OF 32

04 Variables & Constants

Learner outcome Assign values, update variables, and use conventional constant names.

Prerequisite Lessons 01–03 as appropriate; directly builds on the previous topic

What to discuss on this page

- Assignment with = versus equality comparison with ==
- Names, snake_case, and case sensitivity
- Reassignment and how variables refer to objects
- Simple arithmetic using named values
- Uppercase names as conventional constants (not enforced immutability)
- Avoiding reserved keywords and misleading names

Worked example (starter demonstration)

```
price = 120
quantity = 2
TOTAL_TAX_RATE = 0.12
subtotal = price * quantity
print(subtotal)
print(subtotal * TOTAL_TAX_RATE)
```

Explain / expected behavior Explain the order of assignment, the resulting subtotal (240), and the tax amount (28.8).

Practice, apply, and evaluate

Guided practice Change price and quantity, then predict the two printed values.

Independent challenge Model a small order with unit_price, quantity, discount_rate, and total_cost.

Knowledge checks and answer keys

- Does = compare two values? => No, it assigns a value
- Are uppercase Python names immutable? => No, uppercase is a convention
- What does snake_case look like? => unit_price

Page-specific UI / interactive need Variable-value trace table that updates after learner changes source code.

Embedded lab mode python-run

Exact inside-the-lesson hands-on interaction Learner changes price and quantity in the embedded variables example; Run displays recalculated subtotal and tax; Check uses a stated target.

Inline placement Immediately following the relevant example/discussion, on this same lesson page. Do not make the learner navigate to the standalone playground.

Required completion evidence Working example plus learner practice attempt, application submission or self-check, and quiz result according to the shared completion policy.

BASIC PYTHON / LESSON 05 OF 32

05 Data Types

Learner outcome Recognize basic built-in types and convert values appropriately.

Prerequisite Lessons 01–04 as appropriate; directly builds on the previous topic

What to discuss on this page

- int, float, str, and bool with examples
- type() and why string digits are not integers
- Converting with int(), float(), str(), and bool()
- Arithmetic behavior with integers and floats
- None as absence of a value
- Errors from invalid conversions, such as int("hello")

Worked example (starter demonstration)

```
age_text = "20"
age = int(age_text)
height = 1.72
is_student = True
print(age + 1)
print(type(height).__name__)
print(is_student)
```

Explain / expected behavior Output is 21, float, True on separate lines. Emphasize type() and conversion.

Practice, apply, and evaluate

Guided practice Convert a price string to float and add 5; show the result and its type.

Independent challenge Create a user profile with string, integer, float, Boolean, and None values and label each type.

Knowledge checks and answer keys

- What type is "42"? => str
- What does int("42") produce? => Integer 42
- Can int("abc") succeed? => No, it raises ValueError

Page-specific UI / interactive need Type cards, conversion examples, and an inspect-the-value exercise.

Embedded lab mode python-run

Exact inside-the-lesson hands-on interaction Learner modifies string-to-int/float conversion; Run shows values and type(); a deliberate invalid conversion displays the real exception.

Inline placement Immediately following the relevant example/discussion, on this same lesson page. Do not make the learner navigate to the standalone playground.

Required completion evidence Working example plus learner practice attempt, application submission or self-check, and quiz result according to the shared completion policy.

BASIC PYTHON / LESSON 06 OF 32

06 Operators & Expressions

Learner outcome Use arithmetic, comparison, and Boolean operators with correct precedence.

Prerequisite Lessons 01–05 as appropriate; directly builds on the previous topic

What to discuss on this page

- Arithmetic +, -, *, /, //, %, **
- Comparison ==, !=, <, <=, >, >=
- Boolean and, or, not
- Operator precedence and parentheses
- The difference between / and //
- Realistic expressions such as eligibility checks

Worked example (starter demonstration)

```
a = 7
b = 3
print(a / b)
print(a // b)
print(a % b)
print((a > 5) and (b < 5))
```

Explain / expected behavior Contrast true division, floor division, remainder, and a Boolean expression.

Practice, apply, and evaluate

Guided practice Predict values for 2 + 3 * 4 and (2 + 3) * 4, then run.

Independent challenge Compute a shopping total and determine whether it qualifies for free delivery.

Knowledge checks and answer keys

- What does % return? => Remainder
- What is 2 + 3 * 4? => 14
- What is the result of 7 // 3? => 2

Page-specific UI / interactive need Operator reference matrix and live expression evaluator powered by the actual runner.

Embedded lab mode python-run

Exact inside-the-lesson hands-on interaction Learner edits arithmetic and Boolean expressions; Run displays live results; Check tests integer floor-division and precedence.

Inline placement Immediately following the relevant example/discussion, on this same lesson page. Do not make the learner navigate to the standalone playground.

Required completion evidence Working example plus learner practice attempt, application submission or self-check, and quiz result according to the shared completion policy.

BASIC PYTHON / LESSON 07 OF 32

07 Input & Output

Learner outcome Read textual input, convert it safely, and format output.

Prerequisite Lessons 01–06 as appropriate; directly builds on the previous topic

What to discuss on this page

- print() output and separation between print and input
- input(prompt) always returns a string
- Converting numeric input with int()/float()
- f-strings for readable output
- Handling blank/invalid input at a beginner level
- Difference between interactive terminal input and programmatic test input

Worked example (starter demonstration)

```
name = input("Name: ")
age = int(input("Age: "))
print(f"Hello, {name}. Next year: {age + 1}")
```

Explain / expected behavior For sample inputs Ana and 20, output ends with “Hello, Ana. Next year: 21”.

Practice, apply, and evaluate

Guided practice Use the input panel to provide a name and age; try a non-number age to inspect the error.

Independent challenge Build a rectangle area calculator with width and height input and formatted output.

Knowledge checks and answer keys

- What type does input() return? => str
- Why convert an age input? => For numeric arithmetic
- What does an f-string do? => Inserts expressions into formatted text

Page-specific UI / interactive need Stdin input panel, example inputs, output preview, and an invalid-input hint.

Embedded lab mode python-input

Exact inside-the-lesson hands-on interaction Provide editable code AND sample stdin name/age fields; Run displays the greeting or a real conversion error; learner changes inputs.

Inline placement Immediately following the relevant example/discussion, on this same lesson page. Do not make the learner navigate to the standalone playground.

Accuracy / engineering note The runner needs stdin support or an explicit documented limitation and alternate guided mock.

Required completion evidence Working example plus learner practice attempt, application submission or self-check, and quiz result according to the shared completion policy.

BASIC PYTHON / LESSON 08 OF 32

08 Conditional Statements

Learner outcome Implement decisions with if, elif, else, and Boolean logic.

Prerequisite Lessons 01–07 as appropriate; directly builds on the previous topic

What to discuss on this page

- True/False conditions and indentation
- if, elif, and else control flow
- Comparison operators in branch conditions
- and/or/not for combined conditions
- Boundary cases and ordering of conditions
- Nested conditions only after simple branches are clear

Worked example (starter demonstration)

```
score = 78
if score >= 90:
    print("Excellent")
elif score >= 75:
    print("Passed")
else:
    print("Keep practicing")
```

Explain / expected behavior Score 78 prints Passed; discuss what changes at exactly 75 and 90.

Practice, apply, and evaluate

Guided practice Test scores 74, 75, 89, 90 and record selected branches.

Independent challenge Build a fare-discount decision program with age and membership conditions.

Knowledge checks and answer keys

- What does elif mean? => Else if
- Which branch runs for score 78 here? => Passed
- Can multiple branches of one if/elif/else chain run? => No

Page-specific UI / interactive need Branch-flow diagram that highlights the evaluated and selected path.

Embedded lab mode python-run

Exact inside-the-lesson hands-on interaction Learner changes score to choose all three branches; Run displays actual branch; Check asks for an Excellent result without modifying printed text.

Inline placement Immediately following the relevant example/discussion, on this same lesson page. Do not make the learner navigate to the standalone playground.

Required completion evidence Working example plus learner practice attempt, application submission or self-check, and quiz result according to the shared completion policy.

BASIC PYTHON / LESSON 09 OF 32

09 Loops & Iteration

Learner outcome Repeat work with for/while and control iteration safely.

Prerequisite Lessons 01–08 as appropriate; directly builds on the previous topic

What to discuss on this page

- When repetition is useful
- for with range() and iterables
- while loops with a stopping condition
- break versus continue
- Off-by-one errors and range end exclusivity
- Infinite-loop prevention and iteration limits

Worked example (starter demonstration)

```
for number in range(1, 6):
    if number == 3:
        continue
    print(number)
```

Explain / expected behavior Prints 1, 2, 4, 5; 3 is skipped. Explain range(1, 6).

Practice, apply, and evaluate

Guided practice Create a while countdown from 5 to 1 and ensure it terminates.

Independent challenge Build a multiplication table for a given integer using a loop.

Knowledge checks and answer keys

- Does range(1, 6) include 6? => No
- What does break do? => Exits the nearest loop
- What does continue do? => Skips to the next loop iteration

Page-specific UI / interactive need Loop iteration visualizer, step-through table, and safe execution timeout.

Embedded lab mode python-run

Exact inside-the-lesson hands-on interaction Learner changes range limits and continue/break; Run shows resulting numbers; server/browser runtime stops infinite loops with a timeout.

Inline placement Immediately following the relevant example/discussion, on this same lesson page. Do not make the learner navigate to the standalone playground.

Required completion evidence Working example plus learner practice attempt, application submission or self-check, and quiz result according to the shared completion policy.

BASIC PYTHON / LESSON 10 OF 32

10 Functions

Learner outcome Define reusable functions with parameters, return values, and scope.

Prerequisite Lessons 01–09 as appropriate; directly builds on the previous topic

What to discuss on this page

- def syntax and function calls
- Parameters versus arguments
- return versus print
- Default arguments in simple cases
- Local versus global names; avoid unnecessary global state
- Docstrings and naming readable functions

Worked example (starter demonstration)

```
def rectangle_area(width, height):
    return width * height

result = rectangle_area(4, 3)
print(result)
```

Explain / expected behavior Describe input 4 and 3, return 12, then separate printing from returning.

Practice, apply, and evaluate

Guided practice Turn an existing arithmetic expression into a function that accepts parameters.

Independent challenge Create calculate_total(price, quantity, tax_rate=0.0) with at least three tests.

Knowledge checks and answer keys

- What keyword defines a function? => def
- What does return do? => Sends a value to the caller
- Is a local variable automatically global? => No

Page-specific UI / interactive need Function call visualizer, parameter/argument labels, and test cases.

Embedded lab mode python-run

Exact inside-the-lesson hands-on interaction Learner edits function arguments and return expression; Run shows area; Check validates multiple rectangle dimensions.

Inline placement Immediately following the relevant example/discussion, on this same lesson page. Do not make the learner navigate to the standalone playground.

Required completion evidence Working example plus learner practice attempt, application submission or self-check, and quiz result according to the shared completion policy.

BASIC PYTHON / LESSON 11 OF 32

11 Strings & Text Processing

Learner outcome Access, slice, normalize, and format text.

Prerequisite Lessons 01–10 as appropriate; directly builds on the previous topic

What to discuss on this page

- Immutable string objects and indexing
- Zero-based indexing and negative indexes
- Slicing start:stop:step
- len(), strip(), lower(), upper(), replace(), split(), join()
- Searching with in and find()
- Unicode text and case comparison caveats

Worked example (starter demonstration)

```
message = "  Python Basics  "
clean = message.strip().lower()
print(clean)
print(clean[:6])
print("python" in clean)
```

Explain / expected behavior Prints python basics, python, True. Show indices and slice stop exclusion.

Practice, apply, and evaluate

Guided practice Normalize an entered name using strip() and title() and compare inputs.

Independent challenge Build a simple text analyzer reporting word count and whether a word appears.

Knowledge checks and answer keys

- Can s[0] access the first character? => Yes
- Does slicing include its stop index? => No
- Do string methods usually alter the original string object? => No

Page-specific UI / interactive need Clickable string index strip with selected slice preview.

Embedded lab mode python-run

Exact inside-the-lesson hands-on interaction Learner changes text and slice positions; Run shows cleaned string, slice and membership result; explain indexing visually.

Inline placement Immediately following the relevant example/discussion, on this same lesson page. Do not make the learner navigate to the standalone playground.

Required completion evidence Working example plus learner practice attempt, application submission or self-check, and quiz result according to the shared completion policy.

12 Lists

Learner outcome Create, inspect, change, and iterate through ordered mutable collections.

Prerequisite Lessons 01–11 as appropriate; directly builds on the previous topic

What to discuss on this page

- List literals and zero-based indexing
- append(), insert(), remove(), pop()
- Slicing, membership, and len()
- for loops over elements versus indices
- Mutability and shallow copies versus aliases
- Sorting with sorted() versus list.sort()

Worked example (starter demonstration)

```
fruits = ["mango", "banana"]
fruits.append("apple")
fruits[0] = "papaya"
print(fruits)
print(len(fruits))
```

Explain / expected behavior Outputs a three-item list and length 3. Show each mutation step.

Practice, apply, and evaluate

Guided practice Add, replace, and remove shopping list items and show the resulting list.

Independent challenge Implement a grade tracker that calculates average and identifies the highest grade.

Knowledge checks and answer keys

- Are lists mutable? => Yes
- What does append() do? => Adds one item to the end
- What is the index of the first list item? => 0

Page-specific UI / interactive need List state timeline and index labels that follow code execution.

Embedded lab mode python-run

Exact inside-the-lesson hands-on interaction Learner appends/removes/replaces list items; Run shows before/after list and length; Check a targeted target-list state.

Inline placement Immediately following the relevant example/discussion, on this same lesson page. Do not make the learner navigate to the standalone playground.

Required completion evidence Working example plus learner practice attempt, application submission or self-check, and quiz result according to the shared completion policy.

BASIC PYTHON / LESSON 13 OF 32

13 Tuples & Sets

Learner outcome Choose appropriate collection types and apply set operations.

Prerequisite Lessons 01–12 as appropriate; directly builds on the previous topic

What to discuss on this page

- Tuple syntax and immutability of item positions
- Tuple packing/unpacking
- Sets: unique elements and no index-based access
- Union
- , intersection &, difference -
- Membership tests and deduplication
- How tuples, lists, and sets differ

Worked example (starter demonstration)

```
coordinates = (14.6, 121.0)
tags = {"python", "data", "python"}
other = {"data", "web"}
print(coordinates[0])
print(len(tags))
print(tags & other)
```

Explain / expected behavior The unique set has two elements; intersection contains data. Note set display order is not guaranteed.

Practice, apply, and evaluate

Guided practice Deduplicate a list of category names by converting it to a set.

Independent challenge Compare common skills between two teams with union and intersection.

Knowledge checks and answer keys

- Can a tuple element be reassigned? => No
- Does a set keep duplicates? => No
- Can a set be accessed by numeric index? => No

Page-specific UI / interactive need Set Venn diagram and collection comparison grid.

Embedded lab mode python-run

Exact inside-the-lesson hands-on interaction Learner edits two sets and prints union/intersection; Check membership mathematically, never require arbitrary set display order.

Inline placement Immediately following the relevant example/discussion, on this same lesson page. Do not make the learner navigate to the standalone playground.

Accuracy / engineering note Do not assert deterministic ordering of printed sets.

Required completion evidence Working example plus learner practice attempt, application submission or self-check, and quiz result according to the shared completion policy.

BASIC PYTHON / LESSON 14 OF 32

14 Dictionaries

Learner outcome Store, retrieve, and update values by key.

Prerequisite Lessons 01–13 as appropriate; directly builds on the previous topic

What to discuss on this page

- Keys and values with dictionary literal syntax
- Index access versus get() with default
- Add/update/delete entries
- Iterating keys, values, and items
- Key uniqueness and hashability
- A dictionary as a record or configuration

Worked example (starter demonstration)

```
student = {"name": "Ana", "grade": 88}
student["grade"] = 91
student["section"] = "A"
print(student["name"])
print(student.get("grade"))
```

Explain / expected behavior Prints Ana and 91. Show how key-based lookup differs from list indexing.

Practice, apply, and evaluate

Guided practice Add an email and update the grade; show default handling for a missing key.

Independent challenge Build a small inventory dictionary mapping item names to stock quantities.

Knowledge checks and answer keys

- What does a dictionary key do? => Identifies an associated value
- What does get() help avoid when a key is absent? => KeyError
- Can a dictionary contain the same key twice as separate entries? => No

Page-specific UI / interactive need Key-value inspector and editable record table.

Embedded lab mode python-run

Exact inside-the-lesson hands-on interaction Learner edits keys/values and uses get(); Run shows record data; Check verifies a newly added key and updated grade.

Inline placement Immediately following the relevant example/discussion, on this same lesson page. Do not make the learner navigate to the standalone playground.

Required completion evidence Working example plus learner practice attempt, application submission or self-check, and quiz result according to the shared completion policy.

BASIC PYTHON / LESSON 15 OF 32

15 Nested Data Structures

Learner outcome Read and transform lists and dictionaries containing other collections.

Prerequisite Lessons 01–14 as appropriate; directly builds on the previous topic

What to discuss on this page

- Lists of dictionaries and dictionaries of lists
- Multi-step indexing such as students[0]["grades"]
- Traversing nested items with loops
- Basic comprehensions for transforming data
- Avoiding missing-key and out-of-range errors
- Choosing a clear record schema for application data

Worked example (starter demonstration)

```
students = [
    {"name": "Ana", "grades": [90, 95]},
    {"name": "Ben", "grades": [80, 85]}
]
for student in students:
    avg = sum(student["grades"]) / len(student["grades"])
    print(student["name"], avg)
```

Explain / expected behavior Show how to locate each nested grades list and compute per-student averages.

Practice, apply, and evaluate

Guided practice Add a third student and read only the second student's first grade.

Independent challenge Create a product catalog list of dictionaries, then filter in-stock items.

Knowledge checks and answer keys

- What is a nested structure? => A collection inside another
- How do you read Ana's first grade? => students[0]["grades"][0]
- Why use loops with nested data? => To process multiple records consistently

Page-specific UI / interactive need Expandable tree viewer, highlighted access paths, and a basic table representation.

Embedded lab mode python-run

Exact inside-the-lesson hands-on interaction Learner changes nested student grades and reruns averaging loop; Check validates computed output and avoids zero-length input.

Inline placement Immediately following the relevant example/discussion, on this same lesson page. Do not make the learner navigate to the standalone playground.

Required completion evidence Working example plus learner practice attempt, application submission or self-check, and quiz result according to the shared completion policy.

INTERMEDIATE PYTHON / LESSON 16 OF 32

16 Modules & Packages

Learner outcome Import reusable code and explain the relationship between modules and packages.

Prerequisite Lessons 01–15 as appropriate; directly builds on the previous topic

What to discuss on this page

- A module as a .py file and a package as a module namespace
- import and from ... import
- Standard-library examples: math, random, statistics
- Module attributes and qualified names
- Create a reusable local helper module
- Avoid shadowing standard module names

Worked example (starter demonstration)

```
import math
from statistics import mean
print(math.sqrt(81))
print(mean([10, 20, 30]))
```

Explain / expected behavior Prints 9.0 and 20; contrast imported module prefix with direct imported function.

Practice, apply, and evaluate

Guided practice Use random.randint() to simulate a die and document the nondeterministic result.

Independent challenge Split a calculator into main.py and helpers.py; import one helper.

Knowledge checks and answer keys

- What is a module? => A Python file of reusable definitions
- Why use import? => To use names defined in another module
- What is a standard library? => Modules included with Python

Page-specific UI / interactive need File-tree tabs showing main.py and helpers.py, plus a module import diagram.

Embedded lab mode multifile

Exact inside-the-lesson hands-on interaction Show main.py and helpers.py tabs to type/import real helper code if file-based runtime exists; otherwise safely run math/statistics example and label multifile task unavailable.

Inline placement Immediately following the relevant example/discussion, on this same lesson page. Do not make the learner navigate to the standalone playground.

Accuracy / engineering note Multifile editing must be available or lesson demonstrates structure with read-only file mockups.

Required completion evidence Working example plus learner practice attempt, application submission or self-check, and quiz result according to the shared completion policy.

17 File Handling

Learner outcome Read and write UTF-8 text files safely.

Prerequisite Lessons 01–16 as appropriate; directly builds on the previous topic

What to discuss on this page

- File paths and text modes r, w, a
- with open(...) and automatic closure
- read(), readline(), and looping over lines
- Encoding="utf-8" and newline handling
- FileNotFoundError and safe path selection
- Avoiding accidental overwrite of existing files

Worked example (starter demonstration)

```
from pathlib import Path
path = Path("notes.txt")
path.write_text("Python practice\n", encoding="utf-8")
print(path.read_text(encoding="utf-8").strip())
```

Explain / expected behavior Writes then reads a simple local text file. Only run in an isolated writable sandbox.

Practice, apply, and evaluate

Guided practice Append a second line to a temporary practice file and read both lines.

Independent challenge Build a text-based reading log supporting add and list operations.

Knowledge checks and answer keys

- What mode may overwrite a file? => w
- Why use with open(...)? => To close the file automatically
- What error occurs for a missing file read? => FileNotFoundError

Page-specific UI / interactive need Virtual filesystem file browser, content preview, reset button, and non-destructive sandbox storage.

Embedded lab mode virtual-file

Exact inside-the-lesson hands-on interaction Show notes.txt in virtual file navigator; learner writes, appends, reads and sees changed file plus stdout; reset wipes only sandbox files.

Inline placement Immediately following the relevant example/discussion, on this same lesson page. Do not make the learner navigate to the standalone playground.

Accuracy / engineering note Do not expose access to the app host filesystem.

Required completion evidence Working example plus learner practice attempt, application submission or self-check, and quiz result according to the shared completion policy.

18 Exception Handling

Learner outcome Catch expected exceptions, explain errors, and use finally/else appropriately.

Prerequisite Lessons 01–17 as appropriate; directly builds on the previous topic

What to discuss on this page

- Exceptions versus syntax errors
- Reading tracebacks and error types
- try / except with specific exception classes
- else and finally roles
- Raising ValueError for invalid program input
- Why bare except and silent swallowing are problematic

Worked example (starter demonstration)

```
try:
    age = int("twenty")
except ValueError:
    print("Please enter a whole number")
finally:
    print("Finished")
```

Explain / expected behavior The conversion raises ValueError; except handles it; finally always runs.

Practice, apply, and evaluate

Guided practice Convert user-supplied text to float with a friendly error instead of a crash.

Independent challenge Create safe_divide(a, b) handling division by zero and testing both branches.

Knowledge checks and answer keys

- Which exception is raised by int("abc")? => ValueError
- What does finally do? => Runs whether or not an exception occurs
- Should errors be silently ignored? => No

Page-specific UI / interactive need Traceback annotator and side-by-side failing/fixed runs.

Embedded lab mode python-run

Exact inside-the-lesson hands-on interaction Learner edits a try/except conversion and causes or resolves ValueError; Run distinguishes handled result from traceback.

Inline placement Immediately following the relevant example/discussion, on this same lesson page. Do not make the learner navigate to the standalone playground.

Required completion evidence Working example plus learner practice attempt, application submission or self-check, and quiz result according to the shared completion policy.

19 Recursion

Learner outcome Trace recursive calls and identify a stopping base case.

Prerequisite Lessons 01–18 as appropriate; directly builds on the previous topic

What to discuss on this page

- Recursion: function calls itself
- Base case and recursive case
- Call stack visualization
- Tracing factorial on a small integer
- Tradeoffs with iterative alternatives
- Recursion depth limits and risk of nontermination

Worked example (starter demonstration)

```
def factorial(n):
    if n <= 1:
        return 1
    return n * factorial(n - 1)

print(factorial(4))
```

Explain / expected behavior Call chain factorial(4) → 4*3*2*1 → 24. Use nonnegative inputs in examples.

Practice, apply, and evaluate

Guided practice Trace the calls for factorial(3), labeling the base case.

Independent challenge Implement recursive sum of a nonnegative integer list with a correct empty-list base case.

Knowledge checks and answer keys

- What stops recursion? => A base case
- What is factorial(4)? => 24
- Can uncontrolled recursion fail? => Yes, e.g., RecursionError

Page-specific UI / interactive need Collapsible call-stack trace with return-value unwinding.

Embedded lab mode python-run

Exact inside-the-lesson hands-on interaction Learner modifies recursive factorial/nested sum example; Run handles valid and base cases; limit recursion/time resources.

Inline placement Immediately following the relevant example/discussion, on this same lesson page. Do not make the learner navigate to the standalone playground.

Required completion evidence Working example plus learner practice attempt, application submission or self-check, and quiz result according to the shared completion policy.

20 Algorithms

Learner outcome Describe, implement, and compare basic searching/sorting algorithms.

Prerequisite Lessons 01–19 as appropriate; directly builds on the previous topic

What to discuss on this page

- An algorithm as a finite step-by-step procedure
- Linear search and binary search assumptions
- Bubble sort or insertion sort for learning
- Big-O intuition: O(n), O(log n), $O(n^2)$
- Correctness and edge cases (empty lists, duplicates)
- Built-in sorted() and when to favor tested library code

Worked example (starter demonstration)

```
def linear_search(items, target):
    for index, value in enumerate(items):
        if value == target:
            return index
    return -1

print(linear_search([3, 7, 9], 7))
```

Explain / expected behavior Outputs 1. Show how a search visits positions until matching or exhausting items.

Practice, apply, and evaluate

Guided practice Trace search for existing and missing values and count comparisons.

Independent challenge Implement insertion sort and compare output with sorted() on several datasets.

Knowledge checks and answer keys

- What is an algorithm? => A defined procedure to solve a problem
- Must binary search input be sorted? => Yes
- What does O(n) roughly indicate? => Work scales linearly with input size

Page-specific UI / interactive need Step-by-step algorithm animation and comparison counter, no fabricated benchmark claims.

Embedded lab mode python-run

Exact inside-the-lesson hands-on interaction Learner changes sorting/search input and runs algorithm to see output; optionally show verified step trace, not animated fiction.

Inline placement Immediately following the relevant example/discussion, on this same lesson page. Do not make the learner navigate to the standalone playground.

Required completion evidence Working example plus learner practice attempt, application submission or self-check, and quiz result according to the shared completion policy.

INTERMEDIATE PYTHON / LESSON 21 OF 32

21 Object-Oriented Programming

Learner outcome Define classes, instantiate objects, and access attributes/methods.

Prerequisite Lessons 01–20 as appropriate; directly builds on the previous topic

What to discuss on this page

- Classes versus instances (objects)
- Attributes store object data
- Methods define object behavior
- self is the instance reference by convention
- Instantiating multiple distinct objects
- When OOP helps model real entities

Worked example (starter demonstration)

```
class Dog:
    def bark(self):
        return "Woof!"

dog = Dog()
print(dog.bark())
```

Explain / expected behavior An instance calls a method and receives Woof!. Constructors are introduced in Lesson 22.

Practice, apply, and evaluate

Guided practice Create a Book class with a describe() method, then instantiate two books.

Independent challenge Model a Student class that exposes a short profile message.

Knowledge checks and answer keys

- What is a class? => A blueprint defining object data/behavior
- What is an instance? => A created object of a class
- What does self conventionally refer to? => The instance

Page-specific UI / interactive need Class-to-instance diagram and simple attribute inspector.

Embedded lab mode python-run

Exact inside-the-lesson hands-on interaction Learner changes class properties, instantiates objects, and calls a method; Run displays object method result.

Inline placement Immediately following the relevant example/discussion, on this same lesson page. Do not make the learner navigate to the standalone playground.

Required completion evidence Working example plus learner practice attempt, application submission or self-check, and quiz result according to the shared completion policy.

22 Constructors

Learner outcome Initialize instance attributes through __init__.

Prerequisite Lessons 01–21 as appropriate; directly builds on the previous topic

What to discuss on this page

- Object creation and initialization lifecycle
- __init__(self, ...) and instance attributes
- Required/default parameters
- Different instances with independent state
- Light input validation on construction
- Why __init__ is an initializer, not strictly the object allocator

Worked example (starter demonstration)

```
class Student:
    def __init__(self, name, grade=0):
        self.name = name
        self.grade = grade

student = Student("Ana", 92)
print(student.name, student.grade)
```

Explain / expected behavior Prints Ana 92. The __init__ method sets attributes on the new instance.

Practice, apply, and evaluate

Guided practice Add a method passed() that checks a grade threshold.

Independent challenge Build a Product class with name, price, and stock, with validation for nonnegative values.

Knowledge checks and answer keys

- What special method initializes instance attributes? => __init__
- Where is instance data stored? => Attributes such as self.name
- Can constructor parameters have defaults? => Yes

Page-specific UI / interactive need Instance creation timeline and editable constructor argument panel.

Embedded lab mode python-run

Exact inside-the-lesson hands-on interaction Learner modifies __init__ arguments/defaults and instantiates two objects; Run prints constructed attributes.

Inline placement Immediately following the relevant example/discussion, on this same lesson page. Do not make the learner navigate to the standalone playground.

Accuracy / engineering note Correct potential misconception: __init__ initializes an already created instance; __new__ participates in object creation.

Required completion evidence Working example plus learner practice attempt, application submission or self-check, and quiz result according to the shared completion policy.

23 OOP Principles

Learner outcome Illustrate encapsulation, inheritance, polymorphism, and abstraction in small designs.

Prerequisite Lessons 01–22 as appropriate; directly builds on the previous topic

What to discuss on this page

- Encapsulation and public interfaces; Python naming conventions
- Inheritance and class reuse
- Method overriding as a form of polymorphism
- Common interfaces and duck typing
- Abstraction: hide implementation details
- Favor simple composition when inheritance adds unnecessary complexity

Worked example (starter demonstration)

```
class Animal:
    def speak(self):
        return "..."

class Cat(Animal):
    def speak(self):
        return "Meow"

print(Cat().speak())
```

Explain / expected behavior Cat overrides speak() and prints Meow. Discuss method dispatch without claiming privacy is enforced by underscores.

Practice, apply, and evaluate

Guided practice Create Dog and Cat subclasses of Animal and call speak() on both in a loop.

Independent challenge Model two payment methods with the same pay(amount) interface and contrasting behavior.

Knowledge checks and answer keys

- What is inheritance? => A class derives behavior from another
- What is method overriding? => Redefining inherited behavior
- Does a leading underscore enforce private access? => No, it is a convention

Page-specific UI / interactive need Inheritance tree and polymorphic method-call cards.

Embedded lab mode python-run

Exact inside-the-lesson hands-on interaction Learner overrides a subclass method and calls both parent/child; Run demonstrates actual polymorphic output.

Inline placement Immediately following the relevant example/discussion, on this same lesson page. Do not make the learner navigate to the standalone playground.

Required completion evidence Working example plus learner practice attempt, application submission or self-check, and quiz result according to the shared completion policy.

INTERMEDIATE PYTHON / LESSON 24 OF 32

24 Advanced Python Tools

Learner outcome Explain iterators, generators, decorators, and context managers at an introductory level.

Prerequisite Lessons 01–23 as appropriate; directly builds on the previous topic

What to discuss on this page

- Iterator protocol iter()/next() and StopIteration
- yield creates a generator of values lazily
- Memory-related motivation without overstated guarantees
- Decorator functions wrap callables
- Context manager protocol and the with statement
- One concise real-world use for each tool

Worked example (starter demonstration)

```
def countdown(start):
    while start > 0:
        yield start
        start -= 1

print(list(countdown(3)))
```

Explain / expected behavior Outputs [3, 2, 1]. yield suspends and resumes the generator.

Practice, apply, and evaluate

Guided practice Build a generator producing even numbers up to a limit.

Independent challenge Create a logging decorator and demonstrate a with context manager using a sandbox text file.

Knowledge checks and answer keys

- What keyword produces values from a generator? => yield
- What happens when an iterator is exhausted? => StopIteration
- What statement commonly uses context managers? => with

Page-specific UI / interactive need Generator execution timeline, iterator next-step control, and decorator flow diagram.

Embedded lab mode python-run

Exact inside-the-lesson hands-on interaction Learner changes generator yield/iterator logic and reruns; Run prints yielded values without assuming all iterator states are inspectable.

Inline placement Immediately following the relevant example/discussion, on this same lesson page. Do not make the learner navigate to the standalone playground.

Required completion evidence Working example plus learner practice attempt, application submission or self-check, and quiz result according to the shared completion policy.

25 Functional Programming

Learner outcome Pass functions as values and transform iterables with functions/comprehensions.

Prerequisite Lessons 01–24 as appropriate; directly builds on the previous topic

What to discuss on this page

- Functions as first-class values
- Small lambda expressions
- map() and filter() semantics
- List comprehensions as an often-clearer alternative
- Pure-function intuition and limited side effects
- Readable code over nested anonymous expressions

Worked example (starter demonstration)

```
numbers = [1, 2, 3, 4]
squares = list(map(lambda n: n * n, numbers))
evens = [n for n in numbers if n % 2 == 0]
print(squares)
print(evens)
```

Explain / expected behavior Outputs [1, 4, 9, 16] and [2, 4]. Compare two ways to express transformations.

Practice, apply, and evaluate

Guided practice Use filter() to select positive numbers, then rewrite with a comprehension.

Independent challenge Transform a list of product records to discounted prices without mutating originals.

Knowledge checks and answer keys

- What is lambda? => A small anonymous function expression
- What does filter() retain? => Elements whose predicate is truthy
- Can functions be passed as arguments? => Yes

Page-specific UI / interactive need Side-by-side transformations displaying original and result lists.

Embedded lab mode python-run

Exact inside-the-lesson hands-on interaction Learner edits map/filter or a comprehension to transform a list; Run displays resultant list; Check uses sample arrays.

Inline placement Immediately following the relevant example/discussion, on this same lesson page. Do not make the learner navigate to the standalone playground.

Required completion evidence Working example plus learner practice attempt, application submission or self-check, and quiz result according to the shared completion policy.

26 Regex & Date/Time

Learner outcome Match simple text patterns and perform calendar-aware datetime operations.

Prerequisite Lessons 01–25 as appropriate; directly builds on the previous topic

What to discuss on this page

- Raw regex strings and re.search/re.findall
- Character classes and quantifiers, with escaping
- Why simple regex does not fully validate arbitrary email addresses
- datetime.date / datetime.datetime / timedelta
- Parsing with strptime and formatting with strftime
- Naive versus timezone-aware datetimes at a conceptual level

Worked example (starter demonstration)

```
import re
from datetime import date, timedelta
text = "Orders: 12 and 35"
print(re.findall(r"\d+", text))
print(date(2026, 10, 27) + timedelta(days=7))
```

Explain / expected behavior Finds 12 and 35; adds seven calendar days to the illustrative date.

Practice, apply, and evaluate

Guided practice Extract hashtags from a string and format a fixed date as YYYY-MM-DD.

Independent challenge Build a deadline checker that reports whole-day difference between two supplied ISO dates.

Knowledge checks and answer keys

- What does re.findall() return? => All nonoverlapping matches as a list
- What is timedelta for? => Representing time differences
- Is a naive datetime timezone-aware? => No

Page-specific UI / interactive need Regex match highlighter and date arithmetic calendar preview.

Embedded lab mode python-run

Exact inside-the-lesson hands-on interaction Learner changes regex pattern and fixed date inputs; Run displays matches/date differences; highlight actual matches only if derived from code.

Inline placement Immediately following the relevant example/discussion, on this same lesson page. Do not make the learner navigate to the standalone playground.

Accuracy / engineering note Keep sample dates explicitly illustrative, not tied to current date.

Required completion evidence Working example plus learner practice attempt, application submission or self-check, and quiz result according to the shared completion policy.

27 Package Management

Learner outcome Understand pip, dependencies, requirements, and virtual environments.

Prerequisite Lessons 01–26 as appropriate; directly builds on the previous topic

What to discuss on this page

- What third-party packages are and how they differ from standard library
- pip commands run in a terminal, not inside Python source
- python -m pip install package pattern
- python -m venv .venv and activation differences
- requirements.txt and reproducibility
- Package trust, version constraints, and dependency conflicts

Worked example (starter demonstration)

```
# Demonstration script AFTER installing a package externally:
import matplotlib
print("Matplotlib import available")
```

Explain / expected behavior The code imports a dependency only when present. Show shell commands as non-executable terminal snippets.

Practice, apply, and evaluate

Guided practice Create a fresh virtual environment locally, install one trusted demo library, and freeze dependencies.

Independent challenge Prepare project setup instructions listing Python requirements and exact install/test commands.

Knowledge checks and answer keys

- What tool installs Python packages? => pip
- What is a virtual environment? => An isolated project environment
- Where is python -m pip install used? => Terminal/shell

Page-specific UI / interactive need Terminal-command cards, OS activation tabs, and dependency tree illustration.

Embedded lab mode terminal-guide

Exact inside-the-lesson hands-on interaction Provide copyable pip/venv commands, OS-specific activation, and a checklist; no simulated shell execution in Python editor. Optional import test only if dependency installed.

Inline placement Immediately following the relevant example/discussion, on this same lesson page. Do not make the learner navigate to the standalone playground.

Accuracy / engineering note If external packages are unsupported in browser runner, provide a read-only terminal simulation and local instructions.

Required completion evidence Working example plus learner practice attempt, application submission or self-check, and quiz result according to the shared completion policy.

28 Database Integration

Learner outcome Connect Python with SQLite and perform safe CRUD basics.

Prerequisite Lessons 01–27 as appropriate; directly builds on the previous topic

What to discuss on this page

- Tables, rows, columns, keys
- SQLite via Python sqlite3 for the introductory implementation
- CREATE TABLE, INSERT, SELECT, UPDATE, DELETE
- Parameterized queries with ? placeholders to avoid SQL injection
- Transactions, commit(), and context management
- Handling connection errors and showing changes in data

Worked example (starter demonstration)

```
import sqlite3
with sqlite3.connect(":memory:") as conn:
    conn.execute("CREATE TABLE items (id INTEGER, name TEXT)")
    conn.execute("INSERT INTO items VALUES (?, ?)", (1, "Notebook"))
    rows = conn.execute("SELECT name FROM items WHERE id = ?", (1,)).fetchall()
    print(rows)
```

Explain / expected behavior Outputs [('Notebook',)]. Database is in memory and resets on each run.

Practice, apply, and evaluate

Guided practice Insert two more records with bound parameters and query by ID.

Independent challenge Build a minimal inventory CRUD example and demonstrate tests for absent records.

Knowledge checks and answer keys

- What does SELECT do? => Retrieves rows
- Why use ? placeholders? => Safely bind values and reduce SQL injection risk
- What does commit do? => Persist transaction changes

Page-specific UI / interactive need SQL table viewer, query/result split view, resettable SQLite sandbox.

Embedded lab mode sqlite

Exact inside-the-lesson hands-on interaction Learner changes parameterized SQLite INSERT/SELECT in isolated in-memory database; Run displays actual rows and table preview if available.

Inline placement Immediately following the relevant example/discussion, on this same lesson page. Do not make the learner navigate to the standalone playground.

Accuracy / engineering note Never concatenate raw user data into SQL statements. Browser execution requires SQLite-capable engine or server isolation.

Required completion evidence Working example plus learner practice attempt, application submission or self-check, and quiz result according to the shared completion policy.

ADVANCED PYTHON / LESSON 29 OF 32

29 APIs & Web Data

Learner outcome Explain HTTP, parse JSON, and handle predictable API outcomes.

Prerequisite Lessons 01–28 as appropriate; directly builds on the previous topic

What to discuss on this page

- APIs as interfaces to external services
- Request/response flow, GET, status codes 200/404/500
- JSON objects and arrays mapping to Python data
- Network errors, timeouts, and response validation
- Basic requests usage where supported
- Use a local mock API response before any live outbound requests

Worked example (starter demonstration)

```
import json
response_text = '{"city": "Manila", "temp_c": 29}'
data = json.loads(response_text)
print(data["city"])
print(data["temp_c"])
```

Explain / expected behavior Parses a mocked JSON response and prints Manila then 29. No external network is required.

Practice, apply, and evaluate

Guided practice Inspect three mock responses: valid success, missing field, and invalid JSON.

Independent challenge Create a weather-data display consuming a configurable mock endpoint with graceful error states.

Knowledge checks and answer keys

- What does JSON represent? => Structured data in text form
- What does HTTP 404 generally mean? => Resource not found
- Should an API client handle timeouts? => Yes

Page-specific UI / interactive need Request → response diagram, expandable JSON explorer, mock endpoint selector, error cases.

Embedded lab mode mock-api

Exact inside-the-lesson hands-on interaction Learner edits json.loads or mock HTTP response handler; Run parses actual in-page fixture, including missing-field and invalid JSON cases.

Inline placement Immediately following the relevant example/discussion, on this same lesson page. Do not make the learner navigate to the standalone playground.

Accuracy / engineering note No production secrets or API tokens in browser code, static lesson content, or source repository.

Required completion evidence Working example plus learner practice attempt, application submission or self-check, and quiz result according to the shared completion policy.

30 Testing & Deployment

Learner outcome Write automated tests and prepare a small Python app for delivery.

Prerequisite Lessons 01–29 as appropriate; directly builds on the previous topic

What to discuss on this page

- Why tests protect expected behavior
- Assertions and unittest or pytest basics
- Happy-path, edge, and error-case tests
- Run tests and interpret failures
- Basic project README, requirements, configuration
- Deployment meaning and safe handoff; no assumption that hosting is configured

Worked example (starter demonstration)

```
def add(a, b):
    return a + b

def test_add():
    assert add(2, 3) == 5

test_add()
print("Test passed")
```

Explain / expected behavior Assert succeeds; prints Test passed. Production test discovery would use unittest/pytest.

Practice, apply, and evaluate

Guided practice Write and run tests for a function including normal and boundary cases.

Independent challenge Package one previous exercise with a README, requirements, tests, and run instructions.

Knowledge checks and answer keys

- What does assert check? => A condition expected to be true
- What is an edge case? => An unusual or boundary input
- Must deployment mean publishing publicly? => No

Page-specific UI / interactive need Test result badges, per-case explanation, sample project folder layout, and deployment checklist.

Embedded lab mode test-runner

Exact inside-the-lesson hands-on interaction Learner edits function or assert tests; Run produces actual pass/fail/exception details. Deployment stays an instructional checklist.

Inline placement Immediately following the relevant example/discussion, on this same lesson page. Do not make the learner navigate to the standalone playground.

Accuracy / engineering note Do not falsely show “deployed” if the feature merely generates package files.

Required completion evidence Working example plus learner practice attempt, application submission or self-check, and quiz result according to the shared completion policy.

31 Python Graphs: Matplotlib

Learner outcome Create labeled line and scatter graphs, and interpret trends.

Prerequisite Lessons 01–30 as appropriate; directly builds on the previous topic

What to discuss on this page

- Graphs represent numeric relationships
- When to use line graphs versus scatter plots
- Matplotlib pyplot basics: figure, plot/scatter, xlabel, ylabel, title
- Legends, grid, axis range, and readability
- Render plots in a supported runtime
- Read the graph: increase, decrease, clusters, outliers (without asserting causation)

Worked example (starter demonstration)

```
import matplotlib.pyplot as plt
x = [1, 2, 3, 4]
y = [2, 5, 4, 7]
plt.plot(x, y, marker="o")
plt.xlabel("Week")
plt.ylabel("Tasks completed")
plt.title("Weekly progress")
plt.show()
```

Explain / expected behavior Demonstrates an annotated line plot. Provide a deterministic sample chart image if runtime plotting unavailable.

Practice, apply, and evaluate

Guided practice Change a data point and observe the line, axis, and interpretation.

Independent challenge Build one line and one scatter plot from sample records, each with readable labels and captions.

Knowledge checks and answer keys

- Which plot shows connected change over ordered points? => Line graph
- Which plot shows paired numeric observations? => Scatter plot
- Why label axes? => To define units and meaning

Page-specific UI / interactive need Chart preview panel with graph type toggle, code editor, and download/export only if actually supported.

Embedded lab mode plot-preview

Exact inside-the-lesson hands-on interaction Learner edits x/y lists and axis labels; Run should show true Matplotlib output if supported and display real errors; otherwise label fixed screenshot/reference and offer local-running instructions.

Inline placement Immediately following the relevant example/discussion, on this same lesson page. Do not make the learner navigate to the standalone playground.

Accuracy / engineering note Matplotlib may require a server-side isolated worker or pre-rendered fallback; do not assume universal in-browser support.

Required completion evidence Working example plus learner practice attempt, application submission or self-check, and quiz result according to the shared completion policy.

32 Python Charts: Seaborn, Plotly & Pandas

Learner outcome Select and create categorical and distribution charts using appropriate libraries.

Prerequisite Lessons 01–31 as appropriate; directly builds on the previous topic

What to discuss on this page

- Chart choice: bar for categories, histogram for distributions, pie sparingly for part-to-whole
- Pandas DataFrame plotting and Matplotlib backend
- Seaborn statistical plots (e.g., histplot, barplot)
- Plotly interactive plotting when installed
- Readable axes, titles, units, legend and accessible labeling
- Interpreting results without cherry-picked scales
- Package/runtime availability and static vs interactive previews

Worked example (starter demonstration)

```
import pandas as pd
import matplotlib.pyplot as plt
sales = pd.DataFrame({"item": ["A", "B", "C"], "count": [8, 12, 5]})
sales.plot.bar(x="item", y="count", legend=False)
plt.ylabel("Units sold")
plt.title("Sales by item")
plt.tight_layout()
plt.show()
```

Explain / expected behavior Creates a labeled Pandas bar chart. Supplement with distinct Seaborn histogram and Plotly bar examples in tabs.

Practice, apply, and evaluate

Guided practice Compare bar versus histogram for two sample datasets; inspect the effect of changing axis labels.

Independent challenge Create a three-chart mini dashboard with an explicit rationale for each chart type.

Knowledge checks and answer keys

- Which plot compares category totals? => Bar chart
- What does a histogram summarize? => Distribution of numeric values
- Can a chart be misleading without units? => Yes

Page-specific UI / interactive need Library tabs (Pandas / Seaborn / Plotly), chart gallery, legend explainer, and actual-runtime or verified fallback.

Embedded lab mode plot-preview

Exact inside-the-lesson hands-on interaction Tabs for Pandas/Seaborn/Plotly with separate editable starters. Run and preview only installed/supported backends; clearly disable unsupported tabs with accurate instructions.

Inline placement Immediately following the relevant example/discussion, on this same lesson page. Do not make the learner navigate to the standalone playground.

Accuracy / engineering note Do not conflate bar plots with histograms; Plotly interactive previews require a supported renderer.

Required completion evidence Working example plus learner practice attempt, application submission or self-check, and quiz result according to the shared completion policy.

06 Optional extension: Python Frameworks (not counted among the 32)

The source presentation includes a sixth Advanced topic, “Python Frameworks,” and duplicates the number 32. Keep this as an OPTIONAL extension until the project owner resolves lesson numbering. Do not silently overwrite Python Charts.

What to discuss

- What a framework provides: conventions, routing, request handling, reusable structure, and integration tools.
- Flask: a lightweight web framework for simple routes and responses.
- Django: a more comprehensive web framework with batteries-included components.
- Project layout, request → route → response, templating and static files at an introductory level.
- Difference between web frameworks and data/plotting libraries; Pandas is not a web framework.
- Security basics: no debug server exposed publicly, safe configuration, secrets not checked into source.

Simple backend example (show only in suitable environment)

```
from flask import Flask
app = Flask(__name__)

@app.get("/")
def home():
    return "Hello from Flask"

# Run using the project's approved local development command.
```

Page needs: request/response route diagram, project-tree example, HTTP result preview if backend environment exists, setup instructions, and an exercise to add /about. Do not attempt to run arbitrary learner web servers inside the main web app. If user approves this as Lesson 33, update titles, progress denominator, content manifest, QA tests, and course counts consistently.

07 Final project page and submission flow

Source: slides 17–19 require Teach → Practice → Apply, an editable Python Playground, and a final application that learners build, test, explain, and submit. Implement the final project as a separate capstone experience, not as a numbered Lesson 33.

Capstone brief shown to learners

Create a small, demonstrable Python application that solves a concrete problem. Recommended choices: inventory manager, grade tracker, personal task organizer, or simple data-analysis report. The learner must use basic Python control flow and collections; at least one Intermediate concept such as files, modules, OOP, or exceptions; and at least one appropriately supported Advanced concept such as SQLite, API processing with mock response, tests, or an accurately labeled graph/chart. Scope must reflect the runtimes actually available in the product.

Required final-project page sections

- Project introduction and selectable example ideas with clear feasibility boundaries.
- Requirements checklist: problem statement, inputs/outputs, meaningful functions, data structure, error case, and test evidence.
- Planning worksheet: problem, target user, inputs, outputs, data model, pseudocode, milestone checklist.
- Code workspace or upload/attach integration only where existing product supports it; autosave clearly labeled if implemented.
- Test plan table: scenario, input, expected output, actual result, pass/fail and fix note.
- Presentation prompts: problem, design choices, walkthrough, limitations, improvement ideas.
- Submission rules, rubric, status confirmation, and teacher/reviewer feedback only when backend genuinely supports them.

Proposed 100-point evaluation rubric (implementation recommendation)

Criterion	Points	What earns credit
Functional correctness	30	Core workflow works on realistic and edge-case inputs
Python concept use	25	Appropriate Basic + Intermediate + Advanced concepts, explained

Code quality	15	Readable functions, names, structure, and useful comments
Testing & error handling	15	Repeatable tests, meaningful edge cases, handled expected failures
Documentation & demo	15	README, run instructions, explanation and walkthrough

Do not claim automatic grading of the capstone unless a secure grader and validated tests exist. Manual rubric is acceptable when identified clearly.

08 Delivery instructions and final response format for Codex

Implementation deliverables

- Working course routes/pages and reusable UI components.
- Seeded or locally declared validated content for all 32 lessons, with answer keys and code demos.
- Actual execution integration where supported; transparent fallback where not.
- Quizzes, completion/progress behavior, and a capstone page consistent with the existing backend.
- Automated validation and clear operational documentation.

Codex must report at the end

1. Repository assumptions verified. 2. Architecture and files changed. 3. List of all 32 lessons and their working routes. 4. Python runtime used and its sandbox limitations. 5. Quiz, progress, file/SQL/plot capability status. 6. Test commands, actual results, unresolved failures. 7. Deferred features and exact reasons. 8. How to launch and validate locally. 9. Security/accessibility notes and any additional setup required.

Definition of done

The user can navigate all 32 substantive lesson pages, read accurate instructions, edit and actually run code INSIDE the lesson where supported (without leaving for a separate playground), complete meaningful practice, receive understandable quiz feedback, track course progress if the repository supports persistence, and access a clear final project flow. All buttons do something truthful; no lesson uses placeholder content. Advanced lessons state runtime prerequisites honestly, and build/tests pass as reported.

09 Reference links and technical accuracy checklist

Official documentation for implementation verification

- Python tutorial and library docs: https://docs.python.org/3/tutorial/ and https://docs.python.org/3/library/
- Python venv: https://docs.python.org/3/library/venv.html; unittest: https://docs.python.org/3/library/unittest.html; sqlite3: https://docs.python.org/3/library/sqlite3.html
- Python Packaging User Guide: https://packaging.python.org/en/latest/
- Matplotlib docs: https://matplotlib.org/stable/; Pandas visualization: https://pandas.pydata.org/docs/user_guide/visualization.html
- Seaborn docs: https://seaborn.pydata.org/; Plotly Python docs: https://plotly.com/python/
- Flask docs: https://flask.palletsprojects.com/; Django docs: https://docs.djangoproject.com/

These official documentation links are reference destinations for verifying version-specific behaviors; they do not mean the repository already has any of these dependencies.

Final technical accuracy checkpoints

- Python is case-sensitive; input() returns strings; / and // are different; slice end indices are excluded.
- Uppercase constants are a convention, not enforced immutability; tuples cannot reassign positions, but nested mutable objects can still mutate.
- Sets have no guaranteed iteration/display order. Match outputs by values, not literal textual ordering, when sets are involved.
- __init__ initializes an instance; do not describe it as the object allocator. Bare except blocks are discouraged.
- Use parameterized SQL with sqlite3. For APIs, validate status, JSON structure, errors, and timeouts. Never embed tokens in learner code.
- In chart lessons, distinguish line/scatter, bar/histogram/pie, and static vs interactive rendering. Always label axes and units.
- Explicitly mark browser limitations for shell commands, package installation, server creation, filesystem access, network, and third-party imports.